

VT LUMBERJACKS

Predictive AI Evaluation

Steven Woodrum Dharaneesh CKV Nishan Pokharel Nick York

CS 5814 Deep Learning · Virginia Tech · Stanford AIMS Predictive AI Evaluation Competition (NeurIPS 2026)

Checkpoint 2 Presentation · November 2026



Meet the Team

Team Leader	Teammate	Teammate	Teammate
Steven - Lead editor - Approach 1 baseline - Approach 2 Semantic PSN-IRT baseline	**Dharaneesh** - Approach 1 model metadata and optimization - Approach 2 custom embedding	**Nishan** - Approach 1 hyper-optimization - Approach 2 scaling	**Nick** - Approach 1 subject name embedding - Checkpoint 2 presentation structure

Trail Map

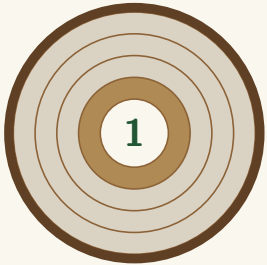
Our presentation broken down

1. The Problem
2. Our Approach
3. Progress So Far
4. What's Next

Questions



Illustration: AI-generated



The Problem

The Motivation

The real world inspiration behind our project

The need

Many organizations, including **schools**, **hospitals**, and **businesses**, increasingly need to assess the suitability of AI systems for their intended use cases, but lack the extensive resources currently required.

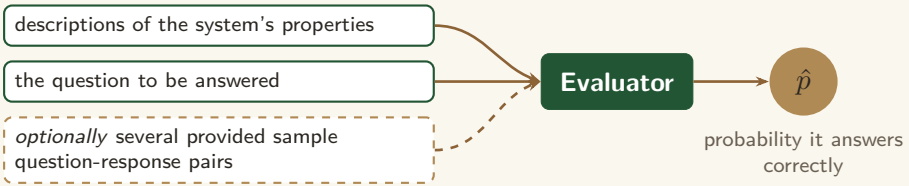


Our goal

Our project has the goal of designing a **reliable evaluator** that can be used to predict the **probability** that a given AI system answers a specific question or task correctly.

The Task

Predict whether an AI system answers an item correctly – without running it



all without explicitly running the AI system on the target question

The Competition

The 2026 Stanford AIMS Predictive AI Evaluation Competition provides **data**, a **concrete objective**, an **evaluation service**, and a **leaderboard** to further motivate this project, and evaluates predictive capacity by testing submitted models on held-out benchmarks.

How We're Scored: Brier ALC

Area under the learning curve – lower is better

Brier score

Brier score is the mean squared difference between predicted probabilities and observed 0 or 1 outcomes. A score of **0 is perfect**; always predicting **0.5 gives 0.25**.

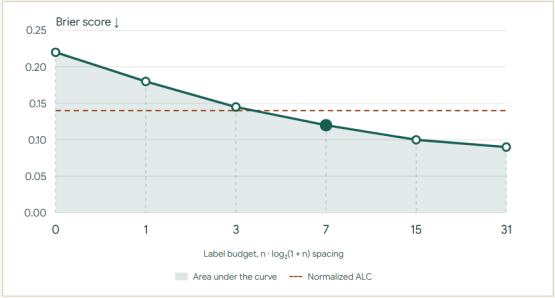
$BS = \frac{1}{N} \sum_{t=1}^{N} (p_t - o_t)^2$

Brier ALC

Brier score is measured at label budgets of 0, 1, 3, 7, 15, and 31.

$$\text{ALC} = 0.1\,B_0 + 0.2\,B_1 + 0.2\,B_3 + 0.2\,B_7 + 0.2\,B_{15} + 0.1\,B_{31}$$

The logarithmic budget axis places greater weight on predictive performance at small label budgets, reflecting the goal of accurate prediction from limited evaluation data.



Illustrative predictor (Brier ALC $= 0.140$). Source: Stanford AIMS, aimslab.stanford.edu/competition

The Data

Competition-provided public training data

The project will use the **AI Measurement Data Bank**, the public training dataset for the 2026 Stanford Predictive AI Evaluation Competition [1]. Containing **six benchmarks**, **12,632 items**, **571,921 responses**, and **543,735 execution traces**.

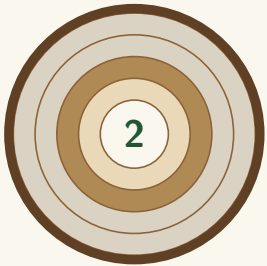
Benchmark	Items	Subjects	Responses	Success rate
matharena	1,755	104	92,944	0.654
mmdocrag	2,000	68	342,695	0.668
multi_swebench	2,126	82	57,975	0.148
real_webagents	233	33	4,449	0.358
researchcodebench	212	31	6,784	0.361
swe_rebench	6,306	1	67,074	0.479
Total	**12,632**		**571,921**	

Terms

An **item** represents a question or task, a **subject** represents an LLM and harness features, and a **response** records the outcome of an attempt.

How we use it

The proposed method will estimate the probability that a subject successfully completes an item using the subject and item information available through the competition interface.



Our Approach

2 of 4

Three Approaches



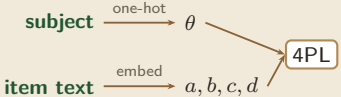




#1 FNN baseline

input → embedding → sigmoid
every team member trains a variant

#2 Two-branch neural IRT



$$P(\text{correct}) = c + \frac{d - c}{1 + e^{-a(\theta - b)}}$$

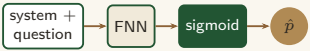
#3 #2 + TTA

same model as #2, plus **test-time adaptation**:
adapts on the labels revealed at each budget

#1 FNN Baseline

How it works

Approach 1 was a basic **binary classifier**. Encoded system metadata and question features were combined and passed through a **feed-forward neural network (FNN)**, with a single **sigmoid output** representing the probability of success. The networks were trained using binary cross-entropy and backpropagation.



One per crew member

Each member trained their own FNN Baseline.

Approach 1 Leave-One-Benchmark-Out Results

Variant	LOBO Brier score ↓ (held-out benchmark)			
	multi_swebench	real_webagents	research-codebench	swe_rebench
1 (Steven)	0.332	0.309	0.329	0.413
2 (Nick)	0.310	0.284	0.311	**0.275**
3 (Dharaneesh)	**0.257**	**0.249**	0.284	0.347
4 (Nishan)	0.279	0.273	**0.253**	0.378

Approach 1 Aggregate Results

Variant	Validation		Test	
	avg Brier	Brier variance	avg Brier ↓	Brier variance
1 (Steven)	0.346	0.0021	0.276	0.0010
2 (Nick)	0.295	0.0003	0.281	0.0050
3 (Dharaneesh)	**0.284**	0.0020	0.238	0.0300
4 (Nishan)	0.296	0.0031	**0.182**	0.0161

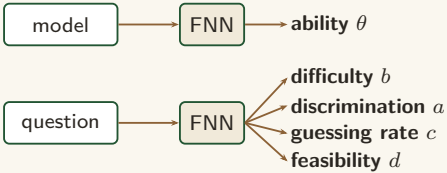
Reference: always predicting 0.5 gives 0.25.

Existing Work: PSN-IRT

Zhou et al., AAAI 2025

What it is

Zhou et al. introduce **PSN-IRT**, a neural Item Response Theory framework for analyzing large language model benchmarks. It uses **separate feed-forward networks** to estimate a model's scalar ability and a question's difficulty, discrimination, guessing rate, and feasibility.



What they used it for

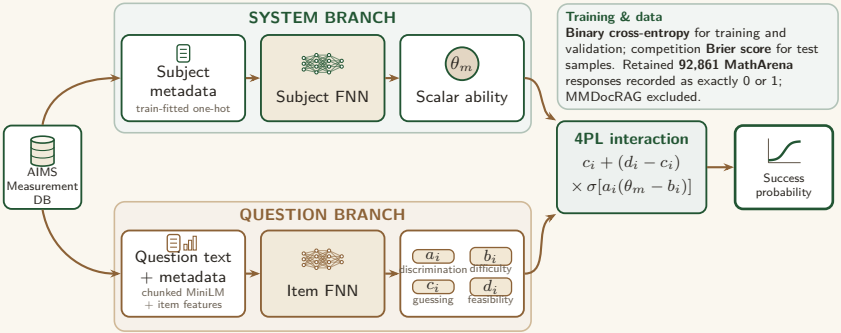
The authors use these estimates to examine **benchmark quality** and construct **smaller evaluation sets**.

How we build on it in approach 2

Reproduce PSN-IRT, but use the **embedding variant** and **our training data**.

#2 Two-Branch Neural IRT

Builds on PSN-IRT



Why not off the shelf

The model only works for a **fixed, known set of items** represented as one-hot input vectors, and forward prediction capability is merely a byproduct of the training process.

What we changed

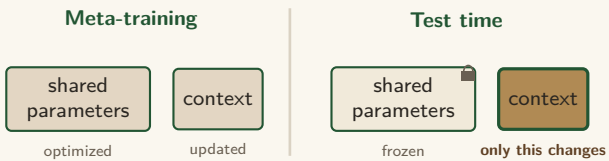
To make the model capable of processing **unseen inputs**, we altered the PSN-IRT architecture to **chunk and semantically embed** natural-language inputs to the item FNN using the same embedding technique from Approach 1.

Existing Work: CAVIA

Zintgraf et al., ICML 2019

What it is

Zintgraf et al. propose **CAVIA**, a meta-learning method that separates **shared model parameters** from a small set of **task-specific context parameters**.



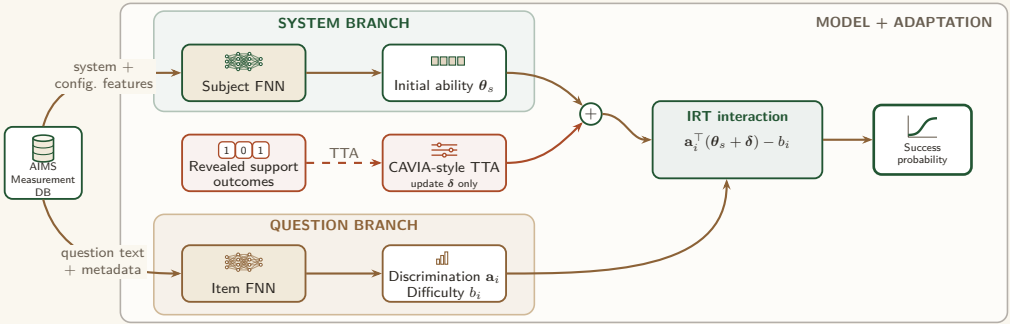
During meta-training, the shared parameters are optimized so that the context parameters can be updated from only a few examples, while at test time **only the context parameters are changed**.

Why it fits Approach 3

This provides a useful foundation for our test-time adaptation stage because it limits adaptation to a small ability correction instead of updating the entire predictor.

#3 Test-Time Adaptation

Use the data revealed at test time



Offline meta-training

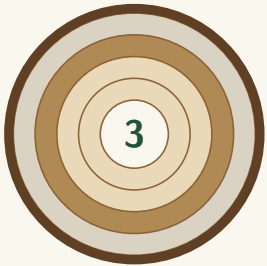
Each episode will contain separate **support** and **query** questions for a subject–benchmark task. Support outcomes will update a small **ability correction vector**, while query loss after adaptation will train the shared networks through that update.

At test time

The networks will remain **fixed** and only the correction will change using revealed support outcomes.

What we'll test

Whether training specifically for adaptation improves predictions beyond the base predictor.

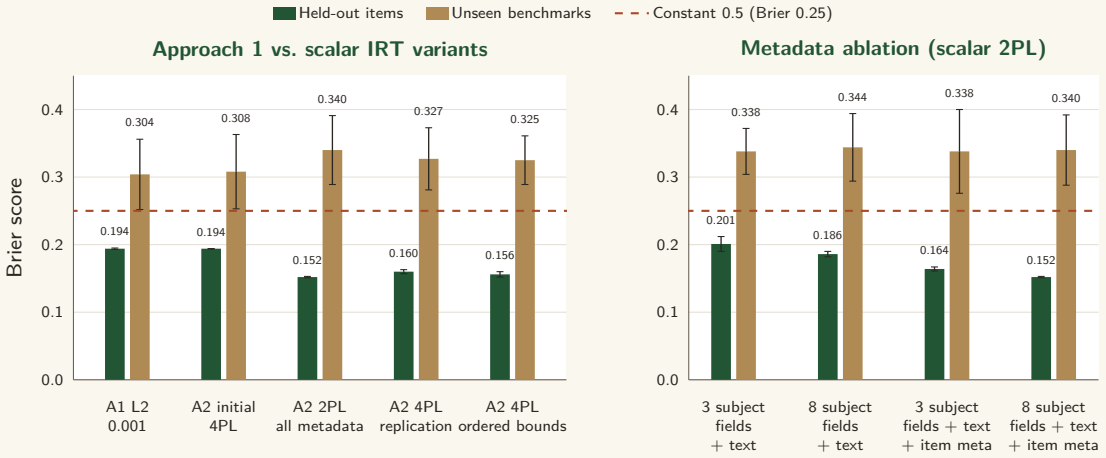


Progress So Far

3 of 4

Results So Far

Equal-pair Brier score – lower is better



Interpretation

- Observation: Not all cross-validation techniques are equal; leave-one-benchmark-out (LOBO) and item-held-out (IHO) CV are both useful when properly interpreted.
  - LOBO scores much worse and higher variance than IHO. **Hypothesis**: Removing approximately 20% of the training data per fold harms performance.
  - IHO scores closely matched competition results. **Hypothesis**: Previously assumed leakage does not occur, and observing semantically similar embeddings across benchmarks is valid and valuable.
- Observation: Understanding the test environment is critical when applying significance to validation results. **Hypothesis**: high loss variance in validation predicts high loss variance in test.

Interpretation cont.

- Observation: Subject and item metadata had a significant effect on predictive performance by adding model-configuration and task-context information, such as reasoning effort, evaluation harness, and programming language. However, tentative test results show similar performance between approaches 1 and 2.
  - **Hypothesis**: Architectures 1 and 2 are more similar than different: feed-forward neural network(s) ingesting embeddings of the same data using the same embedding method, output through an s-shaped activation function ranging from 0 to 1.
  - **Hypothesis**: The addition of test-time adaptation in our final approach will significantly improve performance vs. models that only use historical data.



What's Next

4 of 4

Remaining Work

- Approach 2: Deeper FNNs, vectorized subject ability (θ), formal results from competition.
- Approach 3: Test-time adaptation.
- Final competition results, reproducibility tests, analysis and conclusion.

TIMBER!

Questions?



References

1. Stanford AIMS Lab. *The Predictive AI Evaluation Competition at NeurIPS 2026*. `aimslab.stanford.edu/competition`
2. H. Zhou, H. Huang, Z. Zhao, et al. *Lost in Benchmarks? Rethinking Large Language Model Benchmarking with Item Response Theory* (PSN-IRT). AAAI 2025.
3. L. Zintgraf, K. Shiarlis, V. Kurin, K. Hofmann, S. Whiteson. *Fast Context Adaptation via Meta-Learning* (CAVIA). ICML 2019.
4. S. T. Truong, N. D. Goodman, E. Brunskill, B. Domingue, N. Haber, S. Koyejo. *A Measurement Science Roadmap: From Human Assessment to AI Evaluation*. 2026.
5. S. T. Truong, S. L. Wang, N. Haber, S. Koyejo. *Strategic Evaluation: Incentivizing AI Capability Coverage with Private Benchmarks*. 2026.
6. AIMS. *Empirical Mean Predictor* (PAIEC baseline). `github.com/aims-foundations/paiec_baseline`

Slide design, styling, and diagrams were created with AI assistance. Written content and results are our own.